Notions avancées de visualisation

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Rappel de Matplotlib **4**

A. Tracer une courbe avec Matplotlib 4

III - Notions avancées de Matplotlib **6**

A. Les subplots 6

B. Personnaliser les subplots 8

C. Annotations 11

D. Exporter les graphiques 12

E. Exercice : Quiz 13

IV - Introduction à Seaborn **15**

A. Introduction à Seaborn 15

B. Exercice : Quiz 16

V - L'essentiel **18**

VI - Auto-évaluation **19**

A. Exercice 19

B. Test 19

Solutions des exercices **21**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

- **Durée (en minutes)** : 60
- **Environnement de travail** : Python 3.12+, PyCharm Community
- **Pré requis** : bases en Python, bases en mathématiques (matrices, graphiques), utilisation de base de matplotlib

Contexte

Dans ce cours, nous allons aborder les notions plus avancées de visualisation. Nous commencerons par brièvement rappeler comment nous avons, dans un cours précédent, utilisé les notions de base de la librairie matplotlib pour tracer de simples graphiques.

Nous irons un cran plus loin, par exemple en détaillant les éléments et la façon d'encore mieux personnaliser notre graphique, comment y ajouter des annotations, comment générer des figures avec plusieurs graphiques intégrés (subplots), ou encore comment l'exporter efficacement dans vos documents, articles ou sites webs.

Nous aborderons également quelques notions de la librairie seaborn. Cette librairie, basée sur matplotlib, est plus orientée vers le domaine des statistiques.

Tous droits réservés à STUDI - Reproduction interdite

II Rappel de Matplotlib

A. Tracer une courbe avec Matplotlib

Dans le cours « *Mathématiques en Python* », nous avons déjà abordé la librairie matplotlib. Nous avons créé ensemble trois types de graphiques :

- plt.plot() : il s'agit de simples graphiques linéaires
- plt.pie() : ce sont des graphiques circulaires (« *camemberts* »)
- plt.bar() : ce sont les graphiques en barres verticales

Nous avons également appris comment spécifier le nom des axes, le titre ou encore la légende de notre graphique.

Ci-dessous un petit exemple pour nous rafraichir la mémoire.

Exemple

Le code suivant illustre comment tracer un graphique de type linéaire, contenant deux courbes (nous faisons deux fois appel à la fonction plt.plot()). Les deux courbes tracent respectivement les fonctions mathématiques suivantes :

- $y_1(x) = 2.x$
- $y_2(x) = 3.x$

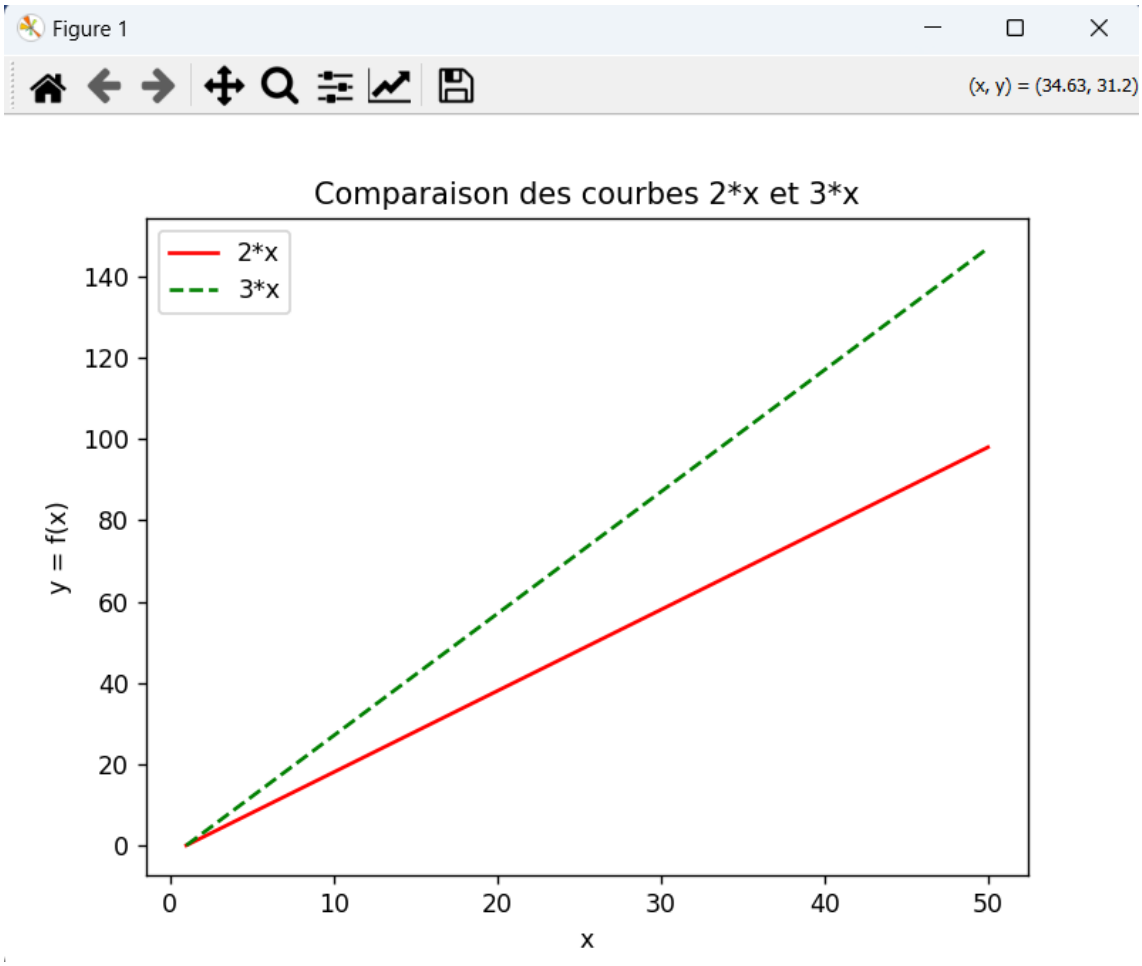
Nous personnalisons le graphique, de sorte à afficher les éléments suivants :

- Légende
- Titre des axes x et y
- Titre du graphique

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(1, 50, 50)
5 y1 = np.ones(50)
6 y2 = np.ones(50)
7 for i in range(0, 50):
8     y1[i]=2*x[i]
9     y2[i]=3*x[i]
10
11 plt.plot(x, y1, 'red', linestyle = '-')
12 plt.plot(x, y2, 'green', linestyle = '--')
13
14 plt.legend(["2*x","3*x"])
15 plt.xlabel("x")
16 plt.ylabel("y = f(x)")
17 plt.title("Comparaison des courbes 2*x et 3*x")
18
19 plt.show()
```


Tous droits réservés à STUDI - Reproduction interdite

Ce code donnera le résultat suivant :



Tous droits réservés à STUDI - Reproduction interdite

III Notions avancées de Matplotlib

A. Les subplots

Lorsqu'on crée une figure (que l'on peut voir comme une fenêtre graphique), on peut y intégrer plusieurs graphiques, un peu à l'image d'un tableau a plusieurs lignes et colonnes. C'est ce qu'on appelle les « *subplots* ». Chaque subplot aura son propre système d'axes. La librairie matplotlib permet, lors de la création d'une nouvelle figure, de spécifier combien de « *cases* », c'est-à-dire combien de graphiques elle devra afficher.

Voyons ceci avec un exemple concret.

Exemple

Dans cet exemple, nous allons repartir de l'exemple précédent, mais nous allons modifier la seconde équation :

- $y_1(x) = 2.x$
- $y_2(x) = e^x$

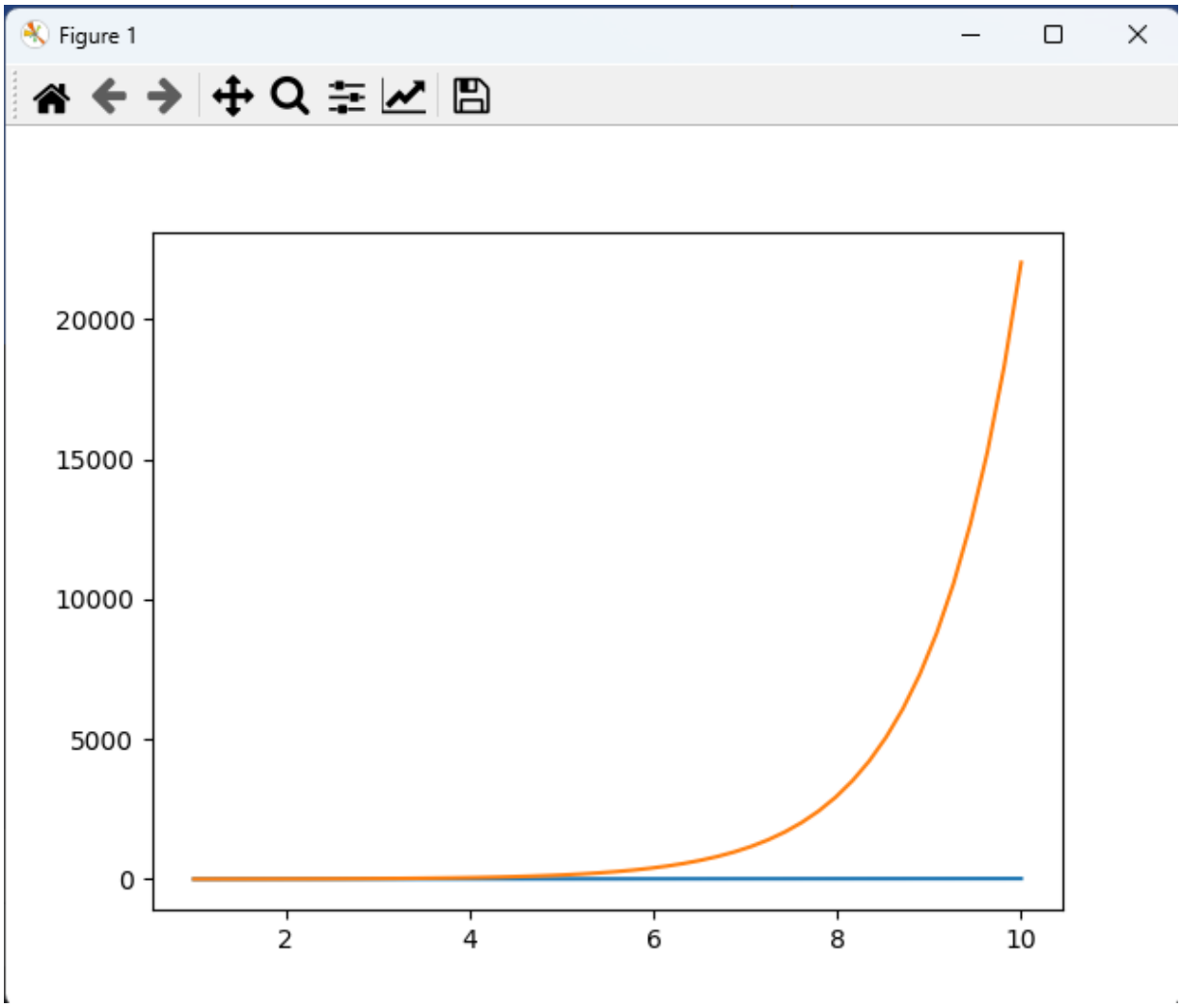
Nous voyons que l'équation y2 représente maintenant la fonction exponentielle.

Si nous traçons ces fonctions sur le même graphique, comme précédemment :

```
import numpy as np
import matplotlib.pyplot as plt
import math

x = np.linspace(1, 10, 50)
y1 = np.ones(x.size)
y2 = np.ones(x.size)

for i in range(x.size):
    y1[i] = 2 * x[i]
    y2[i] = math.exp(x[i])

plt.plot(x,y1)
plt.plot(x,y2)

plt.show()
```


Tous droits réservés à STUDI - Reproduction interdite

Nous obtiendrons le résultat suivant :



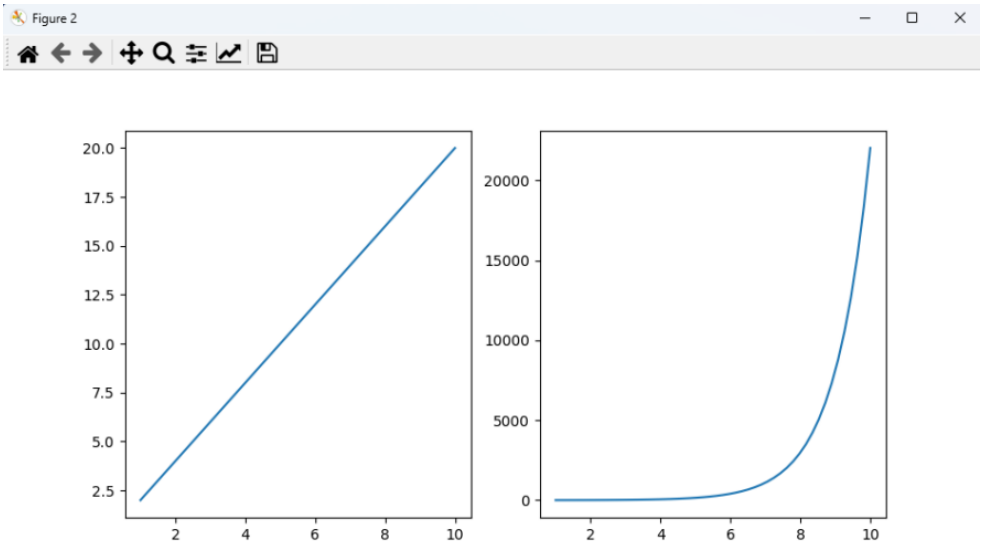
Ce résultat n'est pas très visuel. En effet, les axes nécessaires pour représenter les deux fonctions sont totalement différents. Dans le système d'axes de la fonction exponentielle, la fonction linéaire est complètement écrasée, ce qui rend notre graphique très peu lisible.

Par contre, on pourrait utiliser deux subplots :

```
import numpy as np
import matplotlib.pyplot as plt
import math

x = np.linspace(1, 10, 50)
y1 = np.ones(x.size)
y2 = np.ones(x.size)

for i in range(x.size):
    y1[i] = 2 * x[i]
    y2[i] = math.exp(x[i])

fig, axs = plt.subplots(1,2)  Nous définissons un tableau de 1 ligne,
        2 colonnes

axs[0].plot(x, y1)    Le premier élément de notre fenêtre
axs[1].plot(x, y2)    Le 2eme élément de notre fenêtre
plt.show()
```

Tous droits réservés à STUDI - Reproduction interdite

Ce qui renvoie le résultat suivant :

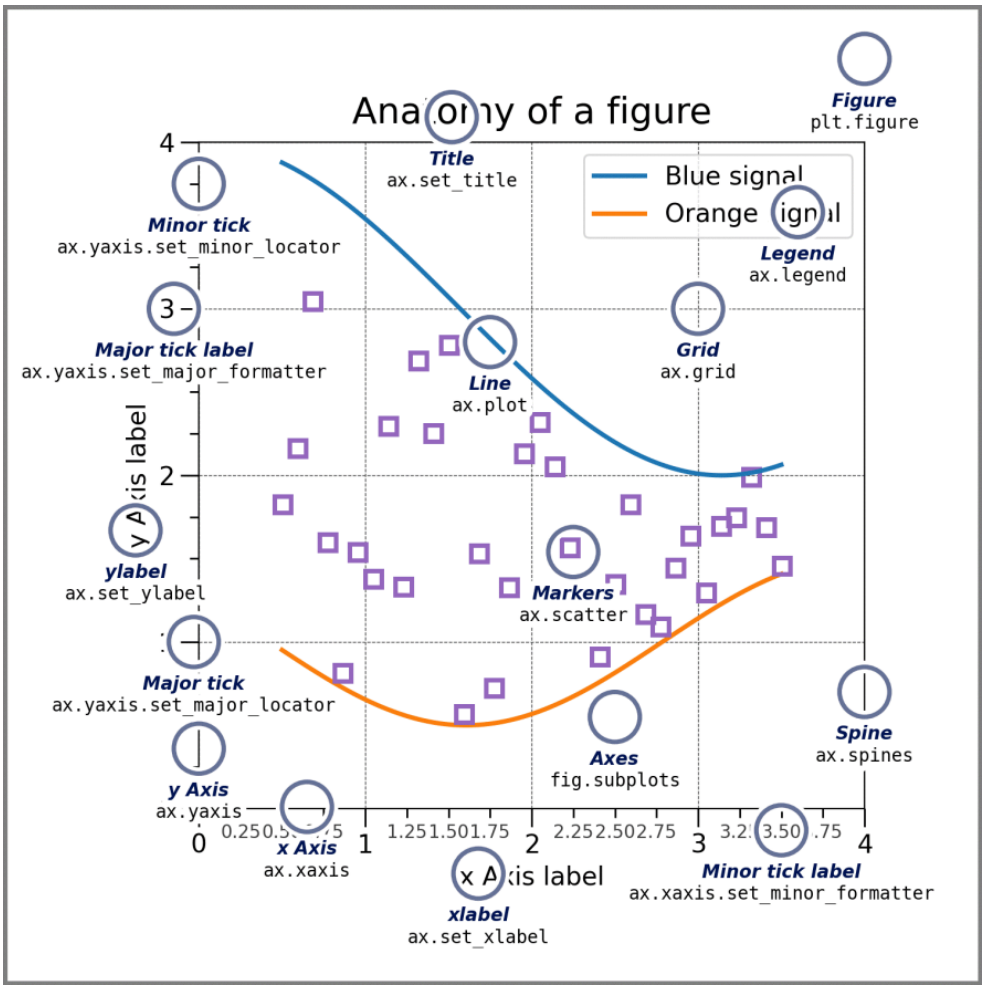


Nous constatons que chaque « *sous graphique* » ou « *sublot* » possède son propre système d'axes. Il est donc tout à fait possible de représenter, cote à cote, des graphiques avec des échelles complètement différentes.

B. Personnaliser les subplots

Étant donné que les subplots possèdent leur propre système d'axes, il est possible de personnaliser chaque axe de manière indépendante. De même, chaque titre peut être personnalisé.

La figure suivante illustre l'ensemble des propriétés qu'il est possible de personnaliser sur un graphique attaché à un système d'axes en particulier.



Tous droits réservés à STUDI - Reproduction interdite

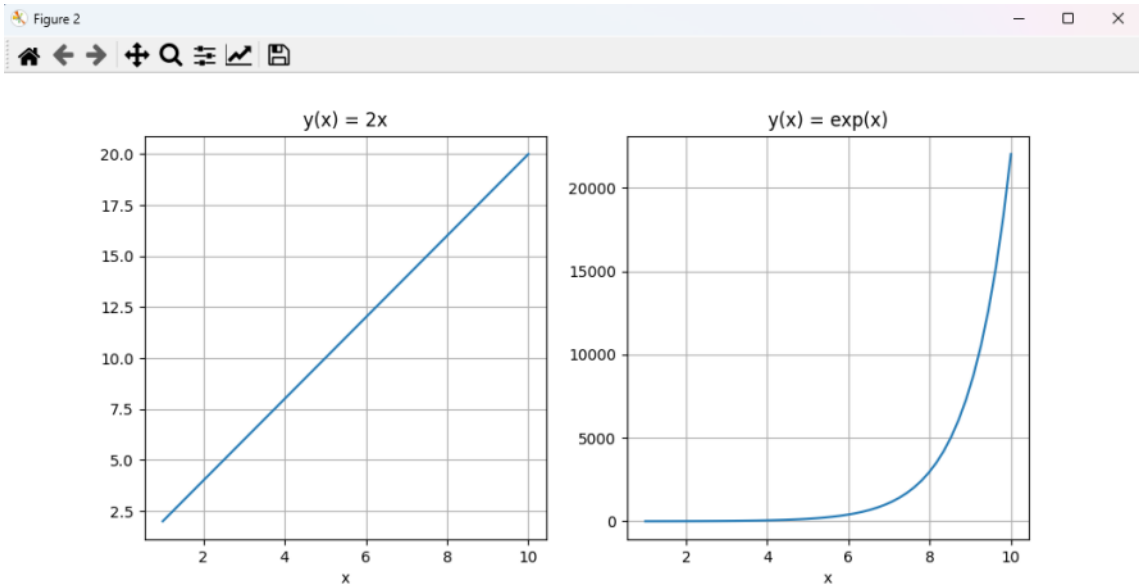
Source : matplotlib[1]

Exemple

Personnalisons le graphique obtenu à l'exemple précédent. Nous allons ajouter un titre à chaque graphique, activer la grille (« *grid* ») et ajouter un titre à l'axe des abscisses.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import math
4
5 x = np.linspace(1, 10, 50)
6 y1 = np.ones(x.size)
7 y2 = np.ones(x.size)
8
9 for i in range(x.size):
10     y1[i] = 2 * x[i]
11     y2[i] = math.exp(x[i])
12
13 plt.plot(x,y1)
14 plt.plot(x,y2)
15
16 fig, axs = plt.subplots(1,2)
17
18 axs[0].plot(x, y1)
19 axs[1].plot(x, y2)
20
21 axs[0].set_title("y(x) = 2x")
22 axs[1].set_title("y(x) = exp(x)")
23
24 axs[0].set_xlabel("x")
25 axs[1].set_xlabel("x")
26
27 axs[0].grid(True)
28 axs[1].grid(True)
29
30 plt.show()
```

Ce qui donne le résultat suivant :



1. https://matplotlib.org/stable/users/explain/quick_start.html#parts-of-a-figure

Tous droits réservés à STUDI - Reproduction interdite

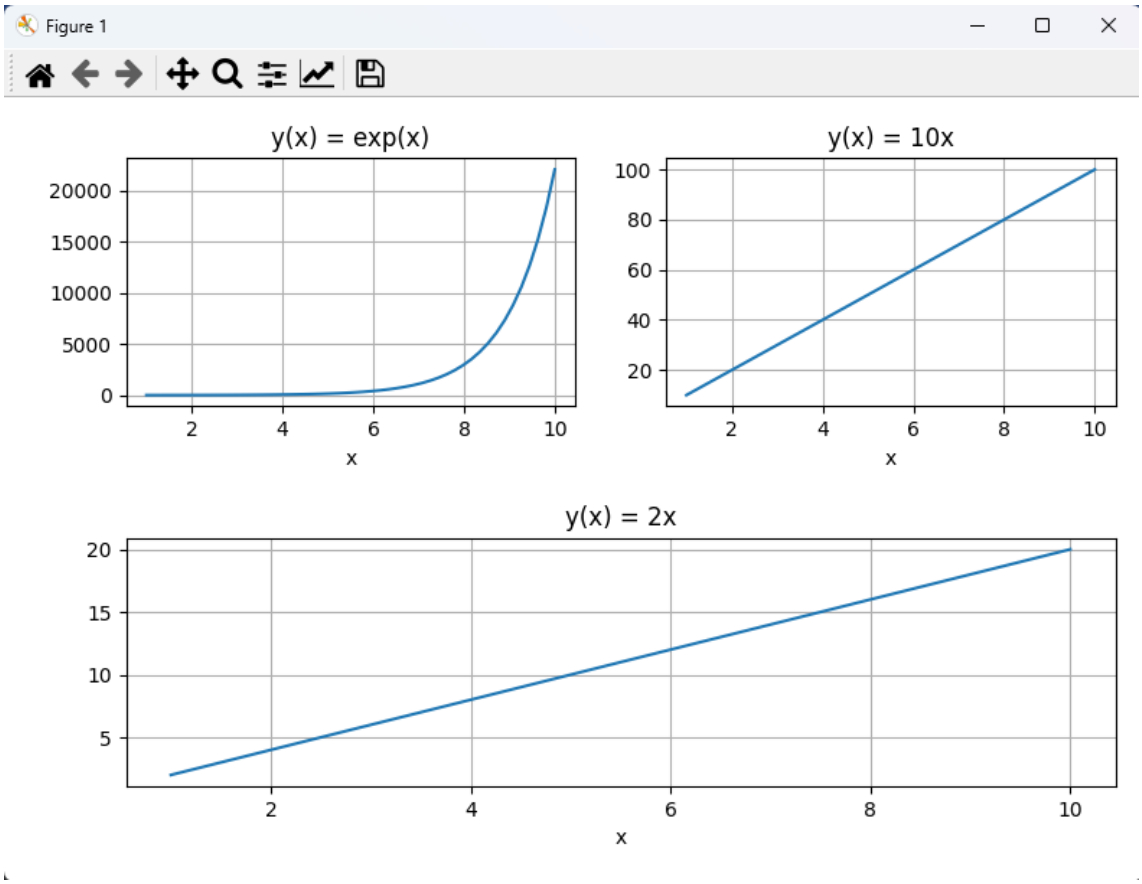
Il est également possible de sélectionner les « *subplots* » et de modifier leur système d'axe directement, sans pour autant utiliser la fonction plt.subplots() (notez l'utilisation du « *s* » à la fin de subplots). Ceci permet par exemple facilement d'étendre un graphique sur plusieurs « *cases* », comme le montre l'exemple suivant :

Exemple

```
import numpy as np
import matplotlib.pyplot as plt
import math

x = np.linspace(1, 10, 50)
y1 = np.ones(x.size)
y2 = np.ones(x.size)
y3 = np.ones(x.size)

for i in range(x.size):
    y1[i] = 2 * x[i]
    y2[i] = math.exp(x[i])
    y3[i] = 10 * x[i]

plt.subplot(2,1,2)
plt.plot(x, y1)
plt.xlabel("x")
plt.grid(True)
plt.title("y(x) = 2x")
plt.tight_layout()

plt.subplot(2,2,1)
plt.plot(x, y2)
plt.xlabel("x")
plt.grid(True)
plt.title("y(x) = exp(x)")
plt.tight_layout()

plt.subplot(2,2,2)
plt.plot(x, y3)
plt.xlabel("x")
plt.grid(True)
plt.title("y(x) = 10x")
plt.tight_layout()

plt.show()
```


Tous droits réservés à STUDI - Reproduction interdite

Ce qui donne le résultat suivant :



Notez ici l'utilisation de plt.tight_layout(), qui demande à maplotlib d'utiliser des marges entre les graphiques de manière à ce qu'ils soient correctement affichés dans des fenêtres de petite taille.

C. Annotations

Il est possible d'annoter nos graphiques en utilisant la fonction annotate(). On y spécifie tout simplement le point de coordonnées à annoter, ainsi que le texte qu'on veut superposer. Reprenons l'exemple précédent, dans lequel on souhaiterait par exemple annoter un point particulier sur le graphique y(x) = 10x :

Exemple

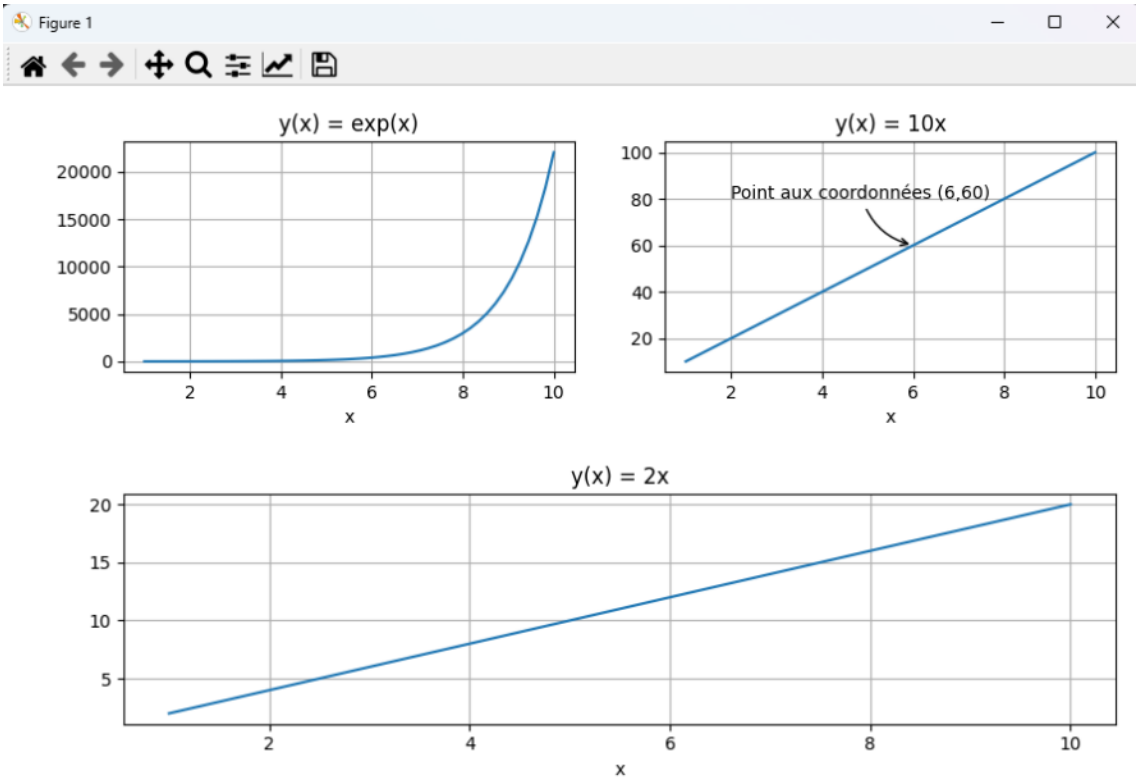
```
import numpy as np
import matplotlib.pyplot as plt
import math

…

plt.subplot(2,2,2)
plt.plot(x, y3)
plt.xlabel("x")
plt.grid(True)
plt.title("y(x) = 10x")
plt.tight_layout()
plt.annotate('Point aux coordonnées (6,60)', xy=(6, 60), xytext=(2, 80),
arrowprops=dict(arrowstyle='->', connectionstyle='arc3, rad=0.3'))


plt.show()
```

Tous droits réservés à STUDI - Reproduction interdite

```
FIN DU LISTING INFORMATIQUE
```

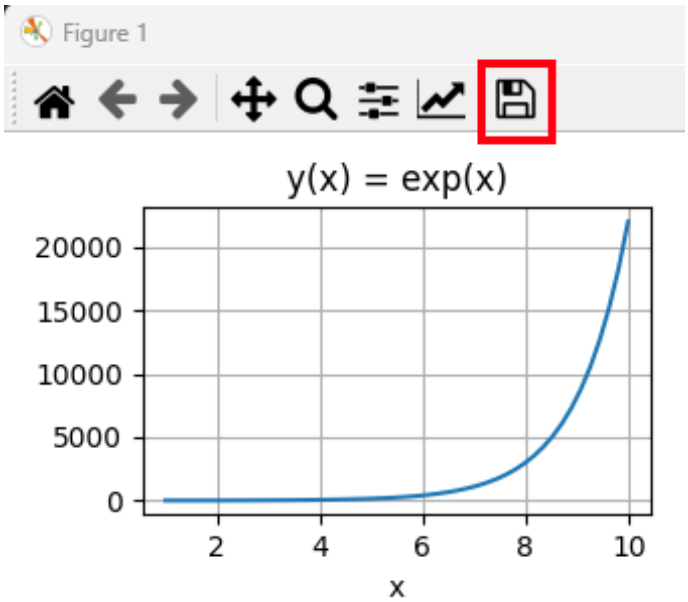
On spécifie ici les propriétés de la flèche qu'on souhaite utiliser : style de flèche, type de connection, rayon de courbure éventuel, etc.).

Ce qui donne le résultat suivant :



D. Exporter les graphiques

Il est possible d'exporter très simplement des graphiques en cliquant sur le bouton « *save* » en haut à droite de votre figure :



Cependant, il est aussi possible d'automatiser l'export des images directement dans le code Python.

Tous droits réservés à STUDI - Reproduction interdite

Pour ce faire, la fonction plt.savefig() peut être utilisée. Il est possible de configurer l'export en précisant éventuellement les arguments suivants :

```
savefig(fname, *, transparent=None, dpi='figure', format=None,
        metadata=None, bbox_inches=None, pad_inches=0.1,
        facecolor='auto', edgecolor='auto', backend=None,
        **kwargs
       )
```

Notons les arguments suivants :

- fname = [string] nom de notre fichier
- dpi = [float] résolution (dots per inch)
- format = [string] extension de fichier ('png', 'pdf', etc.)

E. Exercice : Quiz

[solution n°1 p. 21]

Question 1

Un « *subplot* » c'est :

- Un seul graphique dans une fenêtre
- Un graphique parmi plusieurs graphiques dans une seule et même fenêtre
- Un graphique qui ne s'affiche que dans des conditions bien précises

Question 2

Que fait le code suivant ?

`plt.subplot(1,2,1)`

- Cette commande génère une erreur. La command subplot ne prend que deux arguments
- Elle permet de selectionner, dans une fenetre qui peut contenir deux graphiques (1 rangée de 2 colonnes), la 1ere zone de dessin
- Cette commande génère un graphique qui contient (1,2,1) comme points de données dans l'axe des ordonnées

Question 3

La fonction plt.annotate() permet de…

- Afficher un titre à notre graphique
- Ajouter une étiquette sur un point de coordonnées spécifique de notre graphique
- Ajouter une légende à notre graphique

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Quand utilise-t-on des subplots ?

- **A** Lorsqu'on souhaite afficher conjointement des graphiques qui ont des échelles complètement différentes
- **B** Lorsqu'on souhaite générer des figures de manière compacte dans le cadre de la rédaction d'un document technique
- **C** Pour une simple question de présentation visuelle

Question 5

Si je souhaite afficher la grille sur un graphique, quelle est la commande à utiliser ?

- plt.grid(True)
- plt.setGrid(True)
- plt.enableGrid()

Tous droits réservés à STUDI - Reproduction interdite

IV Introduction à Seaborn

A. Introduction à Seaborn

Seaborn est une librairie basée sur matplotlib, qui est spécialisée dans l'affichage de données statistiques. Elle offre des outils qui simplifient la mise en œuvre de graphiques afin d'extraire de nombreux indicateurs statistiques des données d'entrée.

Seaborn est située à un niveau d'abstraction plus haut que matplotlib, ce qui signifie qu'elle est « *encore plus simple* » à utiliser que matplotlib. Elle contient des fonctions de haut niveau, qui à leur tour font appel à des fonctions plus bas niveau de matplotlib, sans même que l'utilisateur ne s'en rende compte.

Par ailleurs, les palettes de couleurs utilisées par seaborn sont différentes des palettes par défaut de matplotlib, ce qui permet facilement d'identifier les graphiques produits par l'une ou l'autre librairie.

Afin de faciliter la prise en main, seaborn contient plusieurs datasets (« *ensemble de données* ») prêts à être utilisés. Nous allons voir comment on peut très facilement tracer des graphiques depuis les datasets fournis.

Exemple

La librairie seaborn intègre les données sur le crash du Titanic. On peut très simplement en extraire des graphiques statistiques. Notez que pour cet exemple, la librairie 'statsmodels' sera nécessaire afin de calculer une courbe de régression (dans ce cas ci, une régression logistique, puisqu'elle est effectuée sur des données binaires : 1 pour une personne ayant survécu, et 0 sinon) :

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 titanic = sns.load_dataset("titanic")
5
6 print(titanic.head())
7
8 age_bins = [15, 30, 45, 60]
9 fig = sns.lmplot(titanic, x="age", y="survived", hue="sex", x_bins=age_bins,
  logistic=True)
10 fig.set(xlim=(0, 80))
11 plt.show()
```

On commence par charger le dataset « *titanic* » qui contient des données sur le naufrage du titanic, et notamment les âges, sexes, lieux d'embarcation, … des personnes qui étaient à bord du bateau lors de l'accident.

Ensuite, la fonction titanic.head() renvoie les premières lignes contenus dans le dataset. Cela permet de vérifier que le chargement des données s'est bien passé :

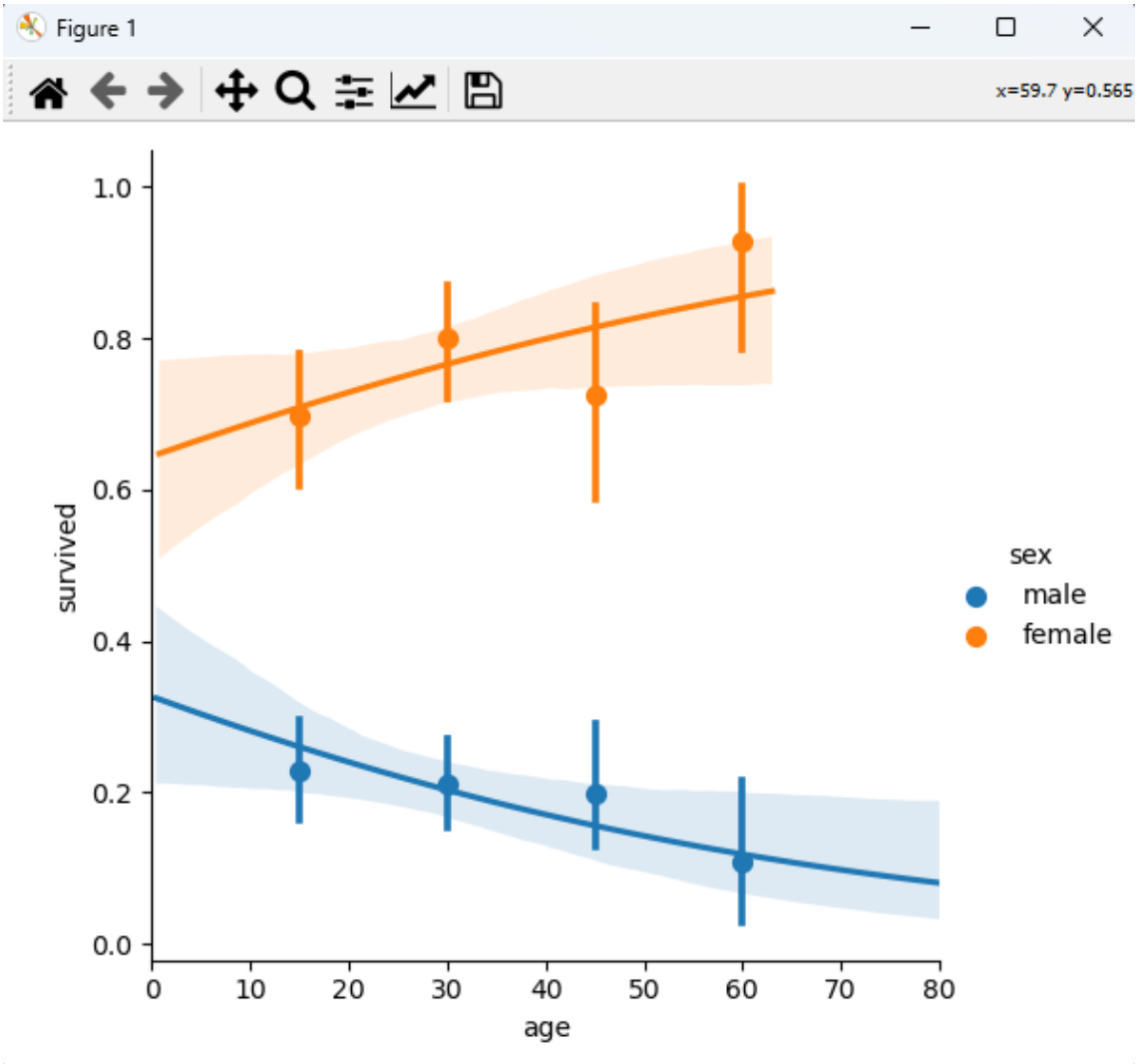
survived	pclass	sex	age	...	deck	embark_town	alive	alone
0	3	male	22.0	...	NaN	Southampton	no	False
1	1	female	38.0	...	C	Cherbourg	yes	False
2	3	female	26.0	...	NaN	Southampton	yes	True
3	1	female	35.0	...	C	Southampton	yes	False
4	3	male	35.0	...	NaN	Southampton	no	True

Tous droits réservés à STUDI - Reproduction interdite

La fonction lmplot() montre que la librairie seaborn, en plus de fournir des fonctions graphiques, est capable d'appliquer des modèles de régression sur des données brutes.

En effet, on voit ici deux courbes orange et bleue, qui sont les régressions sur les données dont la distribution est représentée, pour chaque catégorie d'age définie (15,30,45 et 60 ans), par les barres verticales. La moyenne de ces données est représentée par le point au centre de chaque barre verticale.

Ce qui donne le résultat suivant :



B. Exercice : Quiz

[solution n°2 p. 22]

Question 1

Seaborn, c'est ...

- Une librairie concurrente de matplotlib
- Une librairie complémentaire à matplotlib
- Une base de données

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Seaborn contient uniquement des fonctions d'affichage de graphiques.

- Vrai
- Faux

Question 3

Un dataset c'est …

- Un ensemble d'images
- Un ensemble de données (numériques, visuelles, etc.) répertoriées et triées
- Un tableau Excel

Question 4

Seaborn est une librairie de haut-niveau.

- Vrai
- Faux

Question 5

Seaborn se base sur matplotlib pour tracer des graphiques.

- Vrai
- Faux

Tous droits réservés à STUDI - Reproduction interdite

V L'essentiel

Dans ce cours, nous avons fait un bref rappel de l'utilisation de la librairie matplotlib.

Nous avons vu comment exploiter la notion de subplots. Ceci permet en effet de mettre sur une seule figure, plusieurs graphiques qui peuvent avoir des échelles complètement différentes. Nous avons vu comment personnaliser indépendamment chaque subplot, et nous avons également vu la plupart des champs qu'il est utile de personnaliser afin de rendre un graphique lisible et intelligible.

Nous avons également montré comment nous pouvions annoter des graphiques pour mettre en évidence certains points de données en particulier.

Enfin, nous avons introduit la librairie seaborn, qui se base sur la librairie matplotlib. Seaborn est une librairie à haut niveau d'abstraction, utilisée spécifiquement dans le domaine des statistiques.

Tous droits réservés à STUDI - Reproduction interdite

VI Auto-évaluation

A. Exercice

Supposons que nous souhaitons comparer les fonctions mathématiques suivantes de manière graphique :

- $y_1(x) = sin(x)$
- $y_2(x) = cos(x)$

La correction de cet exercice est disponible en vidéo dans le corrigé de la question 2.

Question 1

[solution n°3 p. 24]

Comment proposez vous de présenter ces fonctions de manière à pouvoir les comparer entre elles ? L'utilisation de subplots est-elle pertinente dans ce cas, et si oui, pourquoi ?

Question 2

[solution n°4 p. 24]

Proposez un code qui trace ces deux fonctions sur une seule et même figure, en les superposant verticalement.

B. Test

Exercice 1 : Quiz

[solution n°5 p. 25]

Question 1

Lorsque je souhaite afficher un graphique, quels éléments est-il important de préciser sur ce graphique ?

- **A** Le titre
- **B** La légende des axes
- **C** L'échelle

Question 2

Seaborn est une librairie qui n'est utilisée que pour des analyses statistiques ?

- Vrai
- Faux

Question 3

Seaborn permet d'exporter plus facilement des images que matplotlib.

- Vrai
- Faux

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Pour exporter mon graphique de manière programmatique, le code suivant est valide :

```
plt.savefig()
```

- Faux
- Vrai

Question 5

Seaborn est native dans Python.

- Vrai
- Faux

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 13]

Question 1

Un « *subplot* » c'est :

- Un seul graphique dans une fenêtre
- ✓ Un graphique parmi plusieurs graphiques dans une seule et même fenêtre
- Un graphique qui ne s'affiche que dans des conditions bien précises

Un subplot c'est un graphique dans une zone géographique précise d'une fenêtre qui en contient plusieurs.

Question 2

Que fait le code suivant ?

```
plt.subplot(1,2,1)
```

- Cette commande génère une erreur. La command subplot ne prend que deux arguments
- ✓ Elle permet de selectionner, dans une fenetre qui peut contenir deux graphiques (1 rangée de 2 colonnes), la 1ere zone de dessin
- Cette commande génère un graphique qui contient (1,2,1) comme points de données dans l'axe des ordonnées

Cette fonction permet de sélectionner un élément (ici le 1er) dans une figure qui peut contenir plusieurs subplots.

Question 3

La fonction plt.annotate() permet de…

- Afficher un titre à notre graphique
- ✓ Ajouter une étiquette sur un point de coordonnées spécifique de notre graphique
- Ajouter une légende à notre graphique

Tous droits réservés à STUDI - Reproduction interdite

plt.annotate() est utilisée pour marquer un point spécifique sur un graphique. On peut y préciser en arguments le texte à afficher ainsi que les paramètres à utiliser pour la flèche qui permet d'identifier le point en question.

Question 4

Quand utilise-t-on des subplots ?

- A Lorsqu'on souhaite afficher conjointement des graphiques qui ont des échelles complètement différentes
- B Lorsqu'on souhaite générer des figures de manière compacte dans le cadre de la rédaction d'un document technique
- C Pour une simple question de présentation visuelle

Dans ce cas-ci, toutes les réponses sont valables. Les subplots ne sont pas uniquement utilisés lorsqu'on souhaite comparer des graphiques avec des échelles différentes, mais aussi simplement pour pouvoir afficher deux graphiques l'un à coté de l'autre, ou même plusieurs graphiques les uns à coté des autres.

Question 5

Si je souhaite afficher la grille sur un graphique, quelle est la commande à utiliser ?

- ✓ plt.grid(True)
- plt.setGrid(True)
- plt.enableGrid()

C'est la commande plt.grid(True) qui permet d'activer la grille sur notre graphique.

Solution n°2

[exercice p. 16]

Question 1

Seaborn, c'est …

- Une librairie concurrente de matplotlib
- ✓ Une librairie complémentaire à matplotlib

Tous droits réservés à STUDI - Reproduction interdite

Une base de données

Seaborn est une librairie, basée sur matplotlib, qui est spécialisée dans l'affichage de résultats statistiques. Il s'agit d'un complément à matplotlib.

Question 2

Seaborn contient uniquement des fonctions d'affichage de graphiques.

Vrai

✓ Faux

En plus de fonctions qui permettent d'afficher des graphiques à orientation statistique, seaborn contient énormément de fonctions mathématiques qui permettent d'extraire des indicateurs des données brutes (par exemple, une régression mathématique).

Question 3

Un dataset c'est …

Un ensemble d'images

✓ Un ensemble de données (numériques, visuelles, etc.) répertoriées et triées

Un tableau Excel

De manière générale, un dataset c'est un ensemble de données d'entrée, qui sont triées par colonnes, lignes, type, etc. Un dataset peut contenir n'importe quel type de données, par exemple numériques ou visuelles (images, etc.)

Question 4

Seaborn est une librairie de haut-niveau.

✓ Vrai

Faux

C'est vrai. Seaborn est une librairie qui fournit un niveau d'abstraction élevé, de manière à en simplifier au maximum l'utilisation.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Seaborn se base sur matplotlib pour tracer des graphiques.

✓ Vrai

Faux

C'est également vrai, même si seaborn « *personnalise* » en interne la façon dont les graphiques sont générés, c'est matplotlib qui tourne en « *arrière plan* ».

[exercice p. 19]

Solution n°3

Les subplots sont intéressants dans ce cas, car superposer verticalement les deux fonctions trigonométriques proposées permettra directement de constater le décalage de 90° (ou de pi/2 radians) entre ces deux fonctions mathématiques.

[exercice p. 19]

Solution n°4

Le code suivant trace les fonctions demandées l'une au dessus de l'autre. On personnalise le titre de nos graphiques et on y ajoute une grille pour pouvoir les comparer plus facilement l'un avec l'autre.

```
import matplotlib.pyplot as plt
import numpy as np
import math

nPoints = 50
xStop = 4*math.pi
x = np.linspace(0, xStop, nPoints)
y1 = np.ones(nPoints)
y2 = np.ones(nPoints)
for i in range(0,nPoints):
    y1[i] = math.sin(x[i])
    y2[i] = math.cos(x[i])

plt.subplot(2,1,1)
plt.plot(x, y1)
plt.grid(True)
plt.title("sin(x)")
plt.tight_layout()

plt.subplot(2,1,2)
plt.plot(x,y2)
plt.grid(True)
plt.title("cos(x)")
plt.tight_layout()

plt.show()
```


Tous droits réservés à STUDI - Reproduction interdite

Solution n°5

[exercice p. 19]

Question 1

Lorsque je souhaite afficher un graphique, quels éléments est-il important de préciser sur ce graphique ?

- **A** Le titre
- **B** La légende des axes
- **C** L'échelle

En réalité, tous ces éléments sont importants. Matplotlib permet très facilement de les paramétrer, et cela permet au lecteur de très facilement comprendre un graphique dans un document.

Question 2

Seaborn est une librairie qui n'est utilisée que pour des analyses statistiques ?

- Vrai
- ✓ Faux

C'est une librairie de data visualisation, qui facilite la création de graphiques clairs et esthétiques.

Question 3

Seaborn permet d'exporter plus facilement des images que matplotlib.

- Vrai
- ✓ Faux

Faux. Seaborn utilise matplotlib, et il est donc tout aussi simple d'exporter des images de graphiques quand ceux si sont générés à l'aide de matplotlib ou seaborn.

Question 4

Pour exporter mon graphique de manière programmatique, le code suivant est valide :

```
plt.savefig()
```

Tous droits réservés à STUDI - Reproduction interdite

✓ Faux

Vrai

Non, il faut absolument spécifier le nom de fichier à utiliser, par exemple :

```
plt.savefig('monGraphe.png')
```

Question 5

Seaborn est native dans Python.

Vrai

✓ Faux

Non, tout comme n'importe quelle librairie externe, il faut installer et ensuite importer la librairie seaborn.

Tous droits réservés à STUDI - Reproduction interdite